

PDDL & PDDL+

Assignment D2-V3:
Domestic Service Robot Mobile Manipulator

Q1: Classical PDDL

Q2: PDDL+

Navigation + Manipulation

A humanoid or mobile manipulator robot must prepare coffee in a kitchen where objects are distributed across multiple locations (e.g., fridge, sink, table). The robot must navigate between locations and manipulate objects.

Mahdi Baghban Ghalehchi — S6741099

Problem & Scenario Overview

Scenario: a mobile manipulator navigates a kitchen graph to collect distributed objects, prepare coffee using functional locations, and serve it at the table.

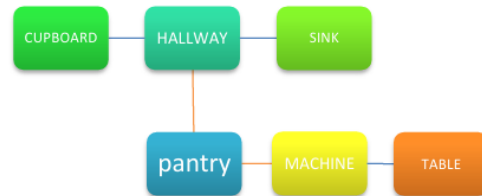
Classical PDDL (Q1)

Simple Layout



Initial state: robot, cup, coffee powder at counter
Goal: served cup1 at the table

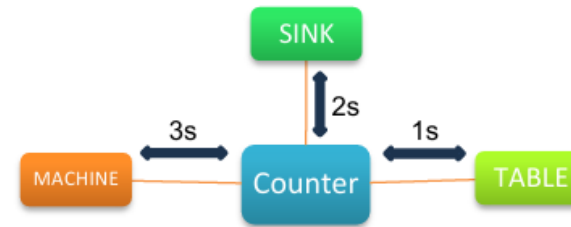
Complex Layout



Initial state: robot at hallway, cup at cupboard, coffee powder at pantry
Goal: served cup1 at the table

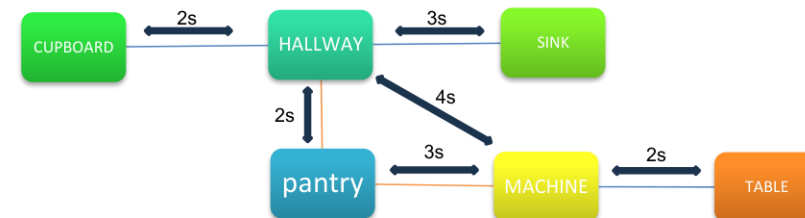
PDDL+ (Q2)

Simple Layout



Initial state: robot, cup, coffee powder at counter
Goal: served cup1 at the table

Complex Layout



Initial state: robot at hallway, cup at cupboard, coffee powder at pantr
Goal: served cup1 at the table

Q1 — Classical PDDL Domain Design

Type	Meaning
location	A discrete place in the kitchen graph, such as counter, sink, machine, or table.
item	A general manipulable object.
cup	A subtype of item representing the coffee cup.
ingredient	A subtype of item representing ingredients such as coffee powder.

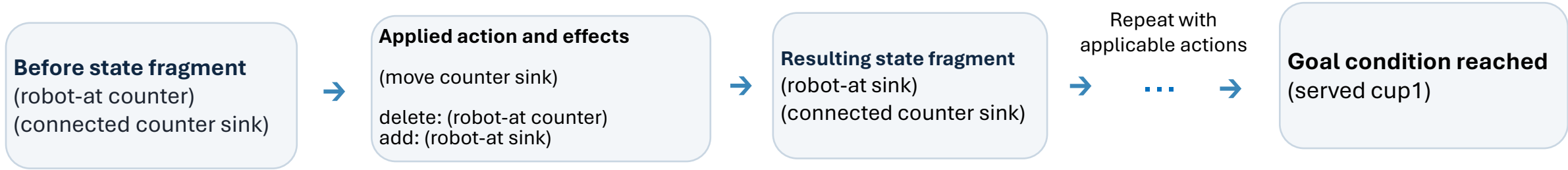
Action group	Actions	Purpose
Navigation	move	Moves the robot between connected locations. It changes only the robot location.
Object handling	pick-up, put-down	Allows the robot to pick and place objects only when it is at the same location as the object.
Coffee preparation	fill-water, add-coffee-powder, brew-coffee, serve-coffee	Represents the symbolic steps required to prepare and serve coffee.

Predicate group	Predicates	Purpose
Navigation	robot-at, connected	Encode the kitchen graph and the current robot location.
Manipulation	handempty, holding, at	Encode object location and gripper capacity.
Kitchen functionality	water-source, coffee-machine, serving-place	Make actions depend on the functional role of the current location.
Coffee state	empty, has-water, has-coffee-powder, brewed, served	Represent the causal preparation chain from empty cup to served coffee.

```
;; Navigation action: changes only the robot position.
(:action move
 :parameters (?from ?to - location)
 :precondition (and
  (robot-at ?from)
  (connected ?from ?to)
 )
 :effect (and
  (not (robot-at ?from))
  (robot-at ?to)
 )
)
```

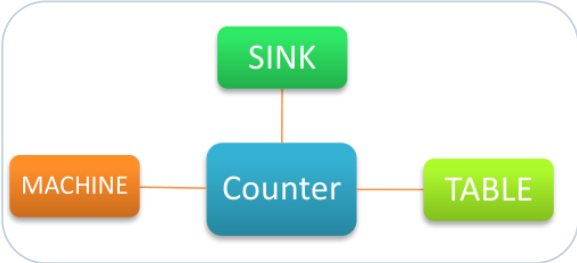
Q1 DESIGN EXAMPLE — State transition for the `move` action

Action: (move counter sink)



Q1 — PDDL Plan Output

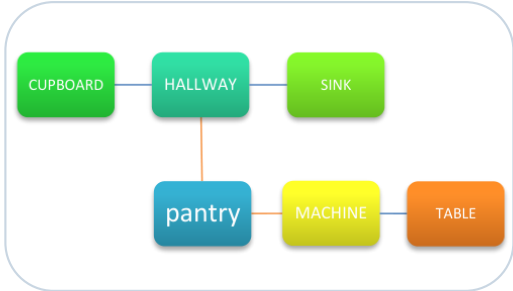
The planner output confirms the intended behaviour: navigation enables location-dependent manipulation, while the distributed objects in the complex layout require additional movement and object-handling steps.



PDDL Plan — Simple Layout

pick-up cup at counter
move to sink → fill-water
return / add powder
move to machine → brew
move to table → serve

#	Action
1	(pick-up cup1 counter)
2	(move counter sink)
3	(fill-water cup1 sink)
4	(move sink counter)
5	(put-down cup1 counter)
6	(pick-up coffee-powder counter)
7	(add-coffee-powder coffee-powder cup1 counter)
8	(put-down coffee-powder counter)
9	(pick-up cup1 counter)
10	(move counter machine)
11	(brew-coffee cup1 machine)
12	(move machine counter)
13	(move counter table)
14	(serve-coffee cup1 table)



PDDL Plan — Complex Layout

start in hallway
retrieve cup from cupboard
go to sink for water
use pantry for coffee powder
direct pantry → machine → table → serve

#	Action
1	(move hallway cupboard)
2	(pick-up cup1 cupboard)
3	(move cupboard hallway)
4	(move hallway sink)
5	(fill-water cup1 sink)
6	(move sink hallway)
7	(move hallway pantry)
8	(put-down cup1 pantry)
9	(pick-up coffee-powder pantry)
10	(add-coffee-powder coffee-powder cup1 pantry)
11	(put-down coffee-powder pantry)
12	(pick-up cup1 pantry)
13	(move pantry machine)
14	(brew-coffee cup1 machine)
15	(move machine table)
16	(serve-coffee cup1 table)

Q2 — PDDL+ Domain Design

PDDL+ replaces instantaneous movement with a timed mechanism: an action starts movement, a process advances progress, and an event completes it.

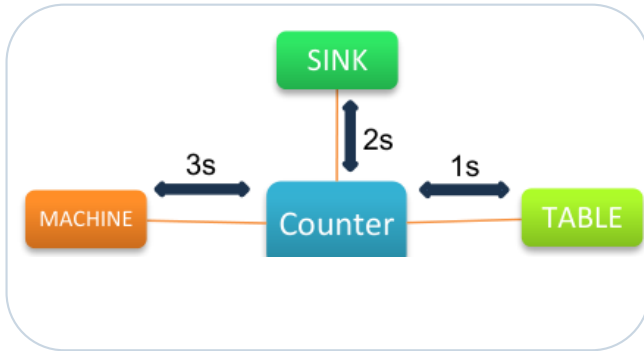
Construct group	Elements	Purpose
Navigation action	<code>`start-move`</code>	Starts time-consuming navigation between connected locations. It activates <code>`moving`</code> , stores the origin and destination, resets <code>`move-progress`</code> , and sets the required movement duration.
Manipulation actions	<code>`pick-up`</code> , <code>`put-down`</code> , <code>`fill-water`</code> , <code>`add-coffee-powder`</code> , <code>`brew-coffee`</code> , <code>`serve-coffee`</code>	Represent the symbolic coffee-preparation steps. These actions remain instantaneous in Q2 because the assignment specifically focuses on time consumption during movement.
Time functions	<code>`elapsed-time`</code> , <code>`deadline`</code>	Store the total mission time and the latest acceptable completion time.
Movement functions	<code>`move-progress`</code> , <code>`current-move-duration`</code> , <code>`move-duration ?from ?to`</code>	Store the progress of the current movement, the required duration of the active movement, and the travel time for each graph edge.
Processes	<code>`mission-clock`</code> , <code>`movement-progress`</code>	<code>`mission-clock`</code> continuously increases <code>`elapsed-time`</code> while the mission is running. <code>`movement-progress`</code> continuously increases <code>`move-progress`</code> while the robot is moving.
Events	<code>`finish-move`</code> , <code>`deadline-violation`</code>	<code>`finish-move`</code> automatically completes movement when progress reaches the required duration. <code>`deadline-violation`</code> automatically marks failure when the deadline is exceeded.

Example movement: (start-move counter sink), with move-duration(counter, sink) = 2 time units



Q2 — PDDL+ Plan Output and Timing

PDDL+ Plan — Simple Layout



Deadline: 12 time units

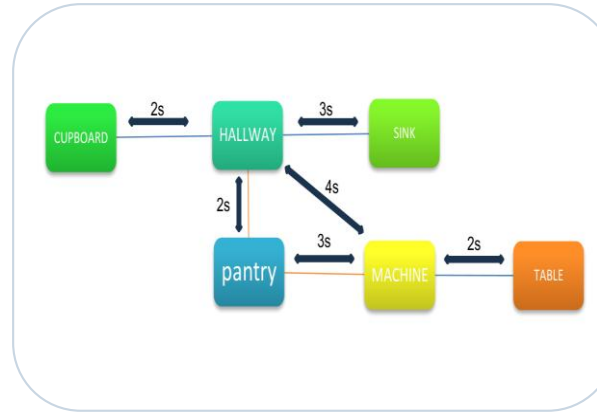
Selected route:

counter → sink → counter →
machine → counter → table

Completion time: $t = 12.0$

Time	Planner output
0.0	(pick-up cup1 counter)
0.0	-----waiting----- [1.0]
1.0	(start-move counter sink)
1.0	-----waiting----- [3.0]
3.0	[event] (finish-move counter sink)
3.0	(fill-water cup1 sink)
3.0	(start-move sink counter)
3.0	-----waiting----- [5.0]
5.0	[event] (finish-move sink counter)
5.0	(put-down cup1 counter)
5.0	(pick-up coffee-powder counter)
5.0	(add-coffee-powder coffee-powder cup1 counter)
5.0	(put-down coffee-powder counter)
5.0	(pick-up cup1 counter)
5.0	(start-move counter machine)
5.0	-----waiting----- [8.0]
8.0	[event] (finish-move counter machine)
8.0	(brew-coffee cup1 machine)
8.0	(start-move machine counter)
8.0	-----waiting----- [11.0]
11.0	[event] (finish-move machine counter)
11.0	(start-move counter table)
11.0	-----waiting----- [12.0]
12.0	[event] (finish-move counter table)
12.0	(serve-coffee cup1 table)

PDDL+ Plan — Complex Layout



Deadline: 18 time units

Selected route:

hallway → cupboard → hallway →
sink → hallway → pantry →
machine → table

Completion time: $t = 18.0$

Time	Planner output
0.0	-----waiting----- [1.0]
1.0	(start-move hallway cupboard)
1.0	-----waiting----- [3.0]
3.0	[event] (finish-move hallway cupboard)
3.0	(pick-up cup1 cupboard)
3.0	(start-move cupboard hallway)
3.0	-----waiting----- [5.0]
5.0	[event] (finish-move cupboard hallway)
5.0	(start-move hallway sink)
5.0	-----waiting----- [8.0]
8.0	[event] (finish-move hallway sink)
8.0	(fill-water cup1 sink)
8.0	(start-move sink hallway)
8.0	-----waiting----- [11.0]
11.0	[event] (finish-move sink hallway)
11.0	(start-move hallway pantry)
11.0	-----waiting----- [13.0]
13.0	[event] (finish-move hallway pantry)
13.0	(put-down cup1 pantry)
13.0	(pick-up coffee-powder pantry)
13.0	(add-coffee-powder coffee-powder cup1 pantry)
13.0	(put-down coffee-powder pantry)
13.0	(pick-up cup1 pantry)
13.0	(start-move pantry machine)
13.0	-----waiting----- [16.0]
16.0	[event] (finish-move pantry machine)
16.0	(brew-coffee cup1 machine)
16.0	(start-move machine table)
16.0	-----waiting----- [18.0]
18.0	[event] (finish-move machine table)
18.0	(serve-coffee cup1 table)

The planner output may show idle waiting intervals to advance time. They are not domain actions, not events, and not manipulation duration.

Reflection & Limitations

Abstraction Gap

- The robot reasons with symbolic facts and discrete locations
- Physical feasibility is abstracted through predicates such as robot-at, at, connected, and holding
- No trajectories, grasp poses, object geometry, or arm configurations are computed
- Real execution would require motion planning, grasp planning, collision checking, and low-level control

Limitations and Known Issues

- Manipulation actions are treated as instantaneous
- Only navigation consumes time
- Waiting intervals only advance planner time; they are not actions, events, or manipulation duration
- Suitable for symbolic task-level planning, not complete physical robot execution

Possible improvements

- Add timed manipulation and connect symbolic planning to motion planning